Object-oriented game simulator Based on game_simulator_v1.py by Turid, NMBU, 2026 Task 1: Updating Character class

In the original version, "attack()" changed the target's prtoected attribute "_hp" directly. This breaks encapsulation, because one object should not modify another object's private data.

To fix this, we added: "Take_damage(amount)" = reduces the character's own HP, burt never below 0. "heal(amount)" = increases the character's own HP, but never above "hp_max". "is_alive" = a read-only property that returns "true" if HP is above 0.

"attack()" now calls "target.take_damage()", so each character is responsible for its own HP.

In [6]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) ->
        """
        Inititialise the Character

        Args:
            name (str): The name of the character
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage dealt when character attacks (default =
        """
        self.name = name
        self._hp = hp_max
        self.hp_max = hp_max
        self.damage = damage

    def take_damage(self, amount: int) -> None:
        """
        Reduce the character's own HP by amount.
        HP cannot end up lower than 0.
        """
        self._hp -= amount
        self._hp = max(self._hp, 0)

    def heal(self, amount: int) -> None:
        """
        Increase the character's own HP by amount.
        HP cannot end up higher than hp_max.
        """
        self._hp += amount
        self._hp = min(self._hp, self.hp_max)

    def attack(self, target) -> None:
        """Attack target character, which then takes damage"""
        target.take_damage(self.damage)

    def introduce(self) -> None:
        """Creates string that introduces character"""
        print(f"Hi, my name is {self.name}!")
```

```
    def __repr__(self) -> str:
        """Representation of character object"""
        return f"Character name={self.name} hp={self.hp} HP and {self.dam

    @property
    def hp(self) -> int:
        """Public read-only property to inspect HP"""
        return self._hp

    @property
    def is_alive(self) -> bool:
        """Read-only property: True if the character has more than 0 HP"""
        return self._hp > 0
```

Testing task 1

In [7]:
```
hero = Character("Hero", hp_max=50, damage=12)
goblin = Character("Goblin", hp_max=30, damage=5)

hero.attack(goblin)
goblin.attack(hero)
print(hero)
print(goblin)

hero.heal(100)
print("After heal:", hero)

hero.attack(goblin)
hero.attack(goblin)
print(goblin, "| alive?", goblin.is_alive)
```

```
Character name=Hero hp=45 HP and 12 damage
Character name=Goblin hp=18 HP and 5 damage
After heal: Character name=Hero hp=50 HP and 12 damage
Character name=Goblin hp=0 HP and 5 damage | alive? False
```

Task 2: The Dice Class

To make damage random, we created "Dice" class that can represent any dice with n sides (n ≥ 2, default 6). "roll()" returns a random number between 1 and the number of sides, using "random,randint". "**repr**" returns a readable string like "Dice(sides=6)". "**Call**" lets us roll the dice by calling the object diretly , so "d20()" gives the same result as "d20.roll()".

If the number of sides is not an integer of at least 2, a "ValueError" is raised.

In [8]:
```
"""
dice.py

The Dice class for battle simulator.
"""

import random

class Dice:
```

```
    """An n-sided dice (n>= 2). Default is a normal 6-sided dice."""

    def __init__(self, sides: int = 6):
        if not isinstance(sides, int) or sides < 2:
            raise ValueError("A dice must have an integer number of sides
        self._sides = sides

    @property
    def sides(self) -> int:
        """Number of sides (read-only)"""
        return self._sides

    def roll(self) -> int:
        """Roll the dice and return a number between 1 and sides."""
        return random.randint(1, self.sides)

    def __call__(self) -> int:
        """Calling the object rolls it: d20() does the same as d20.roll()
        return self.roll()

    def __repr__(self) -> str:
        return f"Dice(sides={self.sides})"
```

Testing task 2

In [9]:
```
d6 = Dice()
d20 = Dice (sides=20)
print (d6, d20)
print(d20.roll(), d20())
```

```
Dice(sides=6) Dice(sides=20)
3 13
```

Discussion of task 1 and 2 results

The test shows that the Character class behaves as expected: After one attack each, the goblin went from 30 to 18 HP and the hero from 50 to 45 HP. Healing the hero by 100 only brought HP back to 50, which confirms that HP cannto exceed "hp_max". After two more attacks the goblin ended at 0 HP instead of a negative value , and "is_alive" returned "False". Trying to set "is_alive" directly gives an "AttributeError", so the property is read-only.

The Dice class also worked: both "d20.roll()" and "d20()" gave numbers between 1 and 20, and "repr" printed "Dice(sides=20)". Because the rolls are random, the numbers will be differently every time the code is run. To get reproducible results, for example when comparing battles later, we could use "random.seed()".

In [ ]:
```

```